

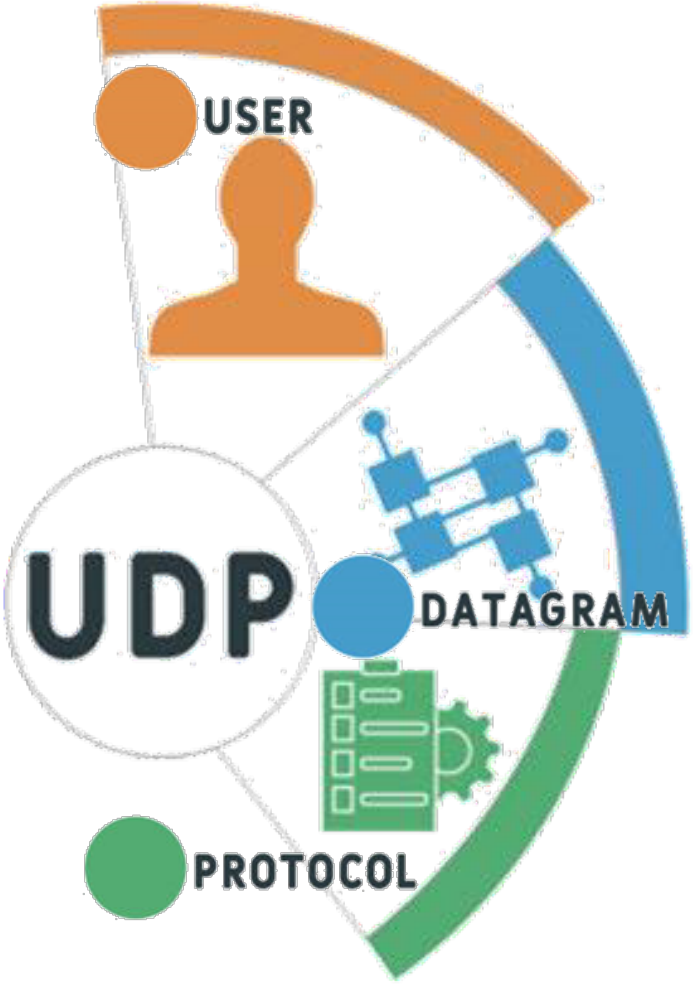
0. INDICE

1. Conceptos previos

2. Clases para sockets UDP

3. Serialización en sockets UDP

4. Seguridad: generación de funciones hash





1. CONCEPTOS PREVIOS

Introducción

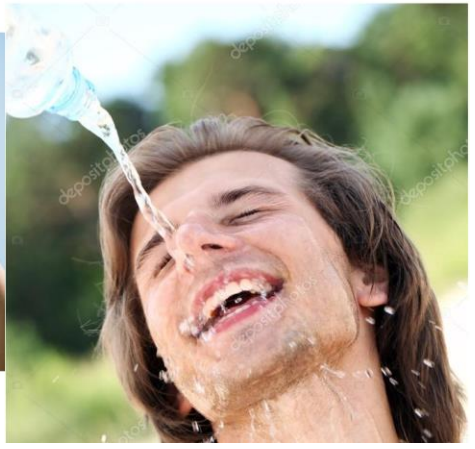
En un socket no orientado a conexión, la comunicación entre las aplicaciones se realiza por medio del protocolo UDP. Esta conexión no es fiable y **no se garantiza que la información enviada llegue a su destino**. Los datagramas se transmiten desde un proceso emisor a otro receptor sin que se haya establecido previamente una conexión, sin acuse de recibo ni reintentos.

Como no se realiza una conexión previa, cada vez que se envía un datagrama, el emisor debe indicar explícitamente la dirección IP y el puerto de destino para cada paquete y el receptor debe extraer la dirección IP y el puerto emisor del paquete.

TCP



UDP





2. CLASES PARA SOCKETS UDP

Clase DatagramPacket (CONSTRUIR DATAGRAMAS)

Los métodos más importantes de esta clase son los siguientes:

MÉTODOS	MISIÓN
DatagramPacket(byte[] buf, int length)	Constructor para **recibir** datagramas: se especifica la cadena de bytes en la que alojar el mensaje (buf) y la longitud de la misma.
DatagramPacket(byte[] buf, int length, InetAddress addrss, int port)	Constructor para **enviar** datagramas: se especifica además la dirección destino y el puerto.
InetAddress getAddress()	Devuelve la IP del host al cual se le envía el datagrama o del que el datagrama se recibió.
byte[] getData()	Devuelve el mensaje contenido en el datagrama tanto recibido como enviado.
int getPort()	Devuelve el número de puerto de la máquina remota a la que se le va a enviar el datagrama o del que se recibió el datagrama.



2. CLASES PARA SOCKETS UDP

Clase DatagramSocket (ENVIAR DATAGRAMAS)

Los métodos más importantes de esta clase son los siguientes:

MÉTODOS	MISIÓN
DatagramSocket(int port)	Construye un socket y lo conecta al puerto local especificado.
DatagramSocket(int port, InetAddress ip)	Constructor que permite especificar, además del puerto local, la dirección local a la que se va asociar el socket.
receive(DatagramPacket paquete)	Recibe un DatagramPacket del socket y lo vuelca a la variable *paquete*.
send(DatagramPacket paquete)	Envía el DatagramPacket *paquete* a través del socket.
int getLocalPort()	Devuelve el puerto en el host local al que está enlazado el socket.
close()	Se encarga de cerrar el socket.



2. CLASES PARA SOCKETS UDP

Ejemplo de conexión entre cliente y servidor

Desde el lado cliente, enviamos un mensaje. Para ello, primero creamos un DatagramPacket y luego un DatagramSocket. Lo hacemos del siguiente modo:

```
public static void main(String[] args) throws IOException {
    int puerto = 12345;
    InetAddress destino = InetAddress.getLocalHost(); //IP host local

    byte[] mensaje = new byte[1024]; //reservamos un espacio de bytes
    String Saludo = "Enviando Saludos!!";
    mensaje = Saludo.getBytes(); //Pasamos el String a bytes

    //Construyo el datagrama a enviar (ojo, el puerto es el puerto destino)
    DatagramPacket envio = new DatagramPacket(mensaje, mensaje.length, destino, puerto);

    //Le asocio un socket (con un puerto de origen) y envio el datagrama
    DatagramSocket socket = new DatagramSocket(34567);
    socket.send(envio);
}
```



2. CLASES PARA SOCKETS UDP

Ejemplo de conexión entre cliente y servidor

Desde el lado servidor, nos quedamos esperando en el puerto 1024 la recepción de datagramas UDP

```
public static void main(String[] args) throws IOException {
    byte[] bufer = new byte[1024];

    //construyo datagrama a recibir
    DatagramPacket recibo = new DatagramPacket(bufer, bufer.length);

    //creamos un socket y recibimos datagrama
    DatagramSocket socket = new DatagramSocket(12345);
    socket.receive(recibo);

    int bytesRec = recibo.getLength(); //obtengo numero de bytes
    String paquete = new String(recibo.getData()); //obtengo String

    System.out.println("Numero de bytes recibidos: "+bytesRec);
    System.out.println("Contenido del paquete: "+paquete.trim());
    System.out.println("Puerto origen del mensaje: "+recibo.getPort());
    System.out.println("IP de origen: "+recibo.getAddress().getHostAddress());
    System.out.println("Puerto destino del mensaje: "+socket.getLocalPort());
    socket.close();
}
```

El socket se queda esperando en este punto…



2. CLASES PARA SOCKETS UDP

Ejemplo de conexión entre cliente y servidor

Lanzamos desde Eclipse el servidor y desde el cmd o bash el Cliente, obteniéndose el siguiente resultado:

```
System.out.println("Numero de bytes recibidos: "+bytesRec);
System.out.println("Contenido del paquete: "+paquete.trim());
System.out.println("Puerto origen del mensaje: "+recibo.getPort());
System.out.println("IP de origen: "+recibo.getAddress().getHostAddress());
System.out.println("Puerto destino del mensaje: "+socket.getLocalPort());
```

Problems @ Javadoc Declaration Console

```
<terminated> Servidor (1) [Java Application] C:\Program Files\JAVA\openjdk-12.0.1_
Numero de bytes recibidos: 18
Contenido del paquete: Enviando Saludos!!
Puerto origen del mensaje: 34567
IP de origen: 192.168.56.1
Puerto destino del mensaje: 12345
```



3. SERIALIZACIÓN EN SOCKETS UDP

Clases ByteArrayOutputStream y ByteArrayInputStream

Para sockets UDP es necesario convertir el objeto a un array de bytes. Imaginemos que queremos convertir un objeto “Persona” a un array de bytes, haríamos lo siguiente:

```
ByteArrayOutputStream bs = new ByteArrayOutputStream();
ObjectOutputStream out = new ObjectOutputStream(bs);
out.writeObject(persona)
byte[] bytes = bs.toByteArray();
```

Y luego, para convertir esos bytes recibidos por el datagrama en un objeto, haríamos:

```
byte[] recibidos = new byte[1024];
ByteArrayInputStream bais = new ByteArrayInputStream(recibidos);
ObjectInputStream in = new ObjectInputStream(bais);
Persona persona = (Persona)in.readObject();
```



4. SEGURIDAD: GENERACIÓN DE FUNCIONES HASH

Resúmenes de mensajes

Los resúmenes de mensajes o funciones hash se usan para generar identificadores de datos únicos y confiables. La clase “MessageDigest” de Java permite a las aplicaciones implementar algoritmos de resumen de mensajes criptográficamente seguros como SHA-256 o SHA-512, lo cual será una garantía de seguridad en el uso de TCP o UDP.

Un digest o hash tiene 2 propiedades:

- Es computacionalmente inviable encontrar 2 mensajes que tengan el mismo valor.
- El digest no debe revelar nada sobre el mensaje original.





4. SEGURIDAD: GENERACIÓN DE FUNCIONES HASH

Clase MessageDigest

MÉTODOS	MISIÓN
MessageDigest.getInstance(algoritmo)	Método constructor: devuelve un objeto MessageDigest que implementa el algoritmo de resumen especificado ("SHA-256", por ejemplo).
md.update(byte[])	Realiza el resumen del array de bytes introducido.
md.digest()	Completa el cálculo del valor hash, devolviendo el resumen obtenido.
md.getDigestLength()	Devuelve la longitud del resumen en bytes,
md.getAlgorithm()	Devuelve el algoritmo de resumen utilizado.
md.getProvider()	Devuelve un objeto de tipo "Provider", que representa a la corporación propietaria del algoritmo criptográfico utilizado (P. ej "SUN").



4. SEGURIDAD: GENERACIÓN DE FUNCIONES HASH

Ejemplo práctico

Para calcular los métodos que nos provee la clase de java MessageDigest, vamos a realizar una práctica donde, dada una cadena de texto, haremos un “update” y un “digest” para calcular su función hash.

Para ello, encerraremos todo el programa “main” en un bloque “try”, capturando la excepción “NoSuchAlgorithmException”, por si no existiera el algoritmo de encriptación que vamos a utilizar.

Comenzamos por instanciar la clase MessageDigest y definiendo la cadena a resumir, que en este caso es un sencillo saludo:

```
md = MessageDigest.getInstance("SHA-256");
String texto = "Hola me llamo Paco Perez";
```



4. SEGURIDAD: GENERACIÓN DE FUNCIONES HASH

Ejemplo práctico

Convertimos la cadena a bytes, y hacemos el “update” y el “digest” en ese órden:

```
//Traducimos el texto a bytes
byte dataBytes[] = texto.getBytes();

//Se introduce texto en bytes a resumir
md.update(dataBytes);

//Se calcula el resumen
byte resumen[] = md.digest();
```

A continuación mostramos información relacionada con el objeto MessageDigest:

```
System.out.println("Mensaje original: "+texto);
System.out.println("Numero de bytes: "+md.getDigestLength());
System.out.println("Algoritmo: "+md.getAlgorithm());
System.out.println("Mensaje resumen: "+new String(resumen));
System.out.println("Mensaje hexadecimal: "+Hexadecimal(resumen));

Provider proveedor = md.getProvider();
System.out.println("Proveedor: " + proveedor.toString());
```



4. SEGURIDAD: GENERACIÓN DE FUNCIONES HASH

Ejemplo práctico

Definimos la siguiente función para poder visualizar el mensaje en Hexadecimal:

```
public static String Hexadecimal(byte[] resumen) {
    String hex = "";
    for (int i=0; i < resumen.length; i++) {
        String h = Integer.toHexString(resumen[i] & 0xFF);
        if (h.length() == 1) hex += 0;
        hex += h;
    }
    return hex.toUpperCase();
}
```

Ejecutamos el programa y obtenemos la siguiente salida:

```
Problems  @ Javadoc  Declaration  Console
<terminated> ejemplo1 [Java Application] C:\Program Files\JAVA\openjdk-12.0.1_windows-x64_bin\jdk-12.0.1\bin\
Mensaje original: Hola me llamo Paco Perez
Numero de bytes: 32
Algoritmo: SHA-256
Mensaje resumen:
□Ïu□&ÔAÑKafÒ□ER•ŠŠ (□P/□ff=,¸/™
Mensaje hexadecimal: 0D11CF751126D441D14B6166D2044552958A8A00280C502F0683663D2CB82F99
Proveedor: SUN version 12
```